

ch4 Data abstraction

C库

`new/new[]` 创建对象或数组

`delete/delete[]` 对应删除

创建之后必须删除，同一对象只能删除一次

C库问题

代码难以理解，需要手动调用函数

The basic object

类中默认private

`::` 是scope resolution operator

每个非静态的成员函数都有一个隐式的this指针

用 `.` 引用对象的成员函数

什么是对象

抽象数据类型

Object details

关于头文件

应该把声明与定义分开

```
1  #ifndef MYHEAD_H //一般和文件名一致
2  #define MYHEAD_H
3  //代码实现
4  #endif
```

为避免命名冲突，不要在头文件中写 `using namespace std`

嵌套结构

可以在类里面定义另一个类

ch5 Hiding the implementation

Setting limits

C++ access control

- `public` 所有人都可访问
- `private` 除了类中成员函数其他人不可访问
- `protected` 与 `private` 类似，但是自己的子类可以访问

Friends

在类中声明友元，其可以访问类的**私有**成员。

普通 *ordinary public member function* 可以：

1. 访问对象的私有部分
2. 函数在类的域里面
3. 含 `this` 指针

静态 *public static member function* 只有前两个

friend function 只有第一个

friend 无法被继承

Global function as a friend

```
1  class Time{
2      friend void show(Time& time);
3  }
4  void show(Time& time){
5      //define
6  }
7  int main(){
8      Time time;
9      show(time);
10 }
```

Global function as friends

```
1  class Boat;
2  class Car{
3      friend int weight(Car& c, Boat& b);
4  }
5  class Boat{
6      friend int weight(Car& c, Boat& b);
7  }
8  int weight(Car& c, Boat& b){
9      return c.weight+b.weight;
10 }
11 void main(){
12     Boat b;
13     Car c;
14     cout<<weight(c,b)<<<<endl;
15 }
```

Member function as a friend

```

1  class Girl;
2  class Boy{
3      void Disp(Girl&); //只声明不定义可以不写变量名
4  }
5  class Girl{
6      friend void Boy::Disp(Girl&);
7  }
8  void Boy::Disp(Girl&x){
9      cout<<name<<endl; //boy的名字
10     cout<<x.name<<endl; //girl的名字
11 }
12 void main(){
13     Boy b;
14     Girl g;
15     b.Disp(g); //输出男孩女孩的名字
16 }

```

A class as a friend

```

1  class X{
2      int x;
3      static int y;
4      friend class Y;
5  public:
6      void set(int i){x=i};
7      void Display(){
8          //输出x和y
9      }
10 };
11 int X::y=1;
12 class Y{
13 public:
14     void set(int i,int j);
15     void display();
16 private:
17     X a; //Y的对象中的a与X创建的对象里面的x是独立的，他们互不影响
18 };
19 void Y::set(int i,int j){
20     a.x=i;
21     X::y=j;
22 }

```

Object layout

The class

member function definition

可以直接在类里面定义，或者在外面。

在外面定义需要用 ::

Partial hiding & handle class

头文件声明，cpp文件实现

ch6 Initialization & cleanup

the constructor

可以被重载，无返回值，含this指针

the destructor

~+类名

不能被重载，一个类只有一个，无返回值

```
1  class Date{
2      int y,m,d;
3      public:
4      Date(int a=0,int b=0,int c=0); //声明：与类名相同
5      ~Date(){cout<<"destructor called."<<endl;}
6  }
7  Date::Date(int a=0,int b=0,int c=0){
8      y=a;
9      m=b;
10     d=c;
11 }
12 void main(){
13     Date d(1,2,3); //自动对对象内的数据赋值
14 }
```

Elimination of the definition block

尽量在用变量近的地方定义变量并初始化

Aggregate initialization

```
1  Date y[3]={Date(1,1,1),Date(2,2,2)};
2  //初始化三次，最后一次全是0
3  Date y[3];
4  y[0]=Date(1,1,1);
5  y[1]=Date(2,2,2); //先默认初始三次均为0，然后依次创建两次临时对象给数组中的赋值，然后立刻
                     //被销毁。一共调用五次构造函数，五次析构函数
```

default constructor

如果自己不定义，编译器会用一个默认的构造函数

ch7 Function Overloading & Default Arguments

More name decoration

相同函数名，不同变量类型或者不同变量的数量称为过载

只返回不同类型不看作过载，只改变变量名称不看作过载

unions

unions和class类似都是存储数据的一个方法，但是unions不能作为基类继承

Default arguments

如果不定义构造函数，那么编译器会生成默认构造函数，默认构造函数可以不提供参数直接引用，默认参数只能从右向左提供

```
1 | int f(int,int=0,char*=0)//ok
2 | int g(int=0,int,char*=0)//error
```

默认参数可以是字符，常量，或者表达式

默认参数在同一个域只能初始化一次

placeholder arguments

`void f2(int x,int,float flt){}` 第二个参数不能被引用，但是引用函数的时候必须提供这个位置的值，如 `f2(1,2,3.0)`

choosing overloading/default arguments

ch8 Constants

Value substitution

`#define` 只做简单的文本替换

`constant` 变量必须初始化，而且后期不能重新赋值

Pointers

```
1 | char s[]={ 'w', 'h', 'o' };
2 | const char* pc=s; //指针指向一个常量，是对指针指向对象的限制
3 | pc[i]='i' //error
4 | char* const cp=s; //指针本身是常量，是对指针的限制
5 | pc='Hi' //error
6 | const char* const cp=s; //指针和指向的字符串都是常量
```

对于一个常量变量，必须用一个指向常量的指针指向它，不是常量的则没有要求

```

1  const float f1=5.0;
2  float*p1=&f1;//error
3  float* const p2=&f1;//error
4  const float* p3=&f1;//ok

```

Function arguments & Return values

```

1  void t(int*p){}
2  void u(const int*cip){
3      *cip=2;//error,这是修改指针指向的对象的常量值
4      int i=*cip;//ok,这是把值传递给另一个对象，没有改变指针对象的值
5  }
6  const char*v(){}
7  const int* const w(){}
8  void main(){
9      int x=0;
10     int *ip=&x;
11     const int *cip=&x;
12     t(ip);//ok,没有任何const限制
13     t(cip);//error
14     u(ip);//ok
15     u(cip);//ok,可以看出函数变量->指针->变量，如果里面那层是const，外面的必须用const
16     char*cp=v();//error
17     const char*ccp=v();//ok
18     int *ip2=w();//error
19     const int* cip2=w();//cip2不是常量也可以
20     //但是只要变量是常量，那么返回值必须返回一个指向常量的指针，指针可以不是常量
21     const int*const cip2=w();
22     *w()=1;//error
23 }

```

Temporaries

编译器自动把所有的临时量设为常量

传递或返回指针或者引用时，把其设为常量

Classes

Static data members

```

1  class MyClass{
2      static int obj;
3  };
4  int MyClass::obj=8;

```

必须被初始化

Const data members

必须用构造函数中的成员初始化列表进行初始化，因此不能用默认的构造函数

```

1  class A{
2      public:
3          A(int i);
4          void Print();
5      private:
6          const int a;
7          static const int b;
8          const int &r;
9  };
10 const int A::b=10;
11 A::A(int i):a(i),r(a){}

```

Const member functions

```

1  class Date{
2      public:
3          int day() const;
4      private:
5          int d;
6  };
7  int day() const{
8      return d;
9  }//在类的外面定义时候也必须加const

```

不能改变类中成员的值，所有对象都可以用

Const objects

对象中的数据成员不会改变值

只能引用常量成员函数

Mutable 可变的

```

1  class Date{
2      public:
3          int year() const{return y++;}
4      private:
5          mutable int y;//这样可以在常量成员函数中改变数据成员
6  }

```

Volatile

意思是可能在编译器未知的情况下数据会变化

和 `const` 语法相同

ch9 Inline Functions

Preprocessor pitfalls

`#define` 只做简单的文本替换

Inline Functions 内联函数

当一个函数很短但是频繁被使用，考虑 `inline`

内联函数必须在被引用之前定义

内联函数不能包含递归，迭代，没有 `switch` 和 `exception`

类中定义的函数默认为内联函数

如果在**类的外面**定义类的成员函数时，必须用 `inline`

Stash & Stack with inlines

Inline & the compiler

Reducing clutter

定义内联函数还是尽量在类外定义

ch10 Name Control

Static elements

对象存储在静态存储区

其他文件无法访问静态对象

Static variables inside functions

```
1 void f(){
2     static n=1;
3     int x=1;
4     cout<<n++<<endl;
5     cout<<x++<<endl;
6 }
7 void main()
8 {
9     f();f();
10 }
```

函数内静态变量内存不会随着函数的结束而释放，值会一直保留，而且只在第一次调用的函数进行一次初始化

Static class objects inside functions

对内置的类型的静态变量如果没有初始化，那么默认初始化为0，但是对用户自创建的类型必须用构造函数初始化

```

1  class X{
2      int i;
3      public:
4      X(int ii=0):i(ii){}
5      ~X(){}
6  };
7  void f(){
8      static X x1(1);
9      static X x2;
10 }
11 void main(){
12     f();
13 }

```

Static object destructors

全局对象的构造函数一般在主函数前引用

函数中的静态对象只有当函数被引用时会被构造

析构函数引用和构造的顺序正好相反

Controlling linkage

- external: 全局变量函数，加 `extern` 后其他文件都能调用
- internal: 静态的全局变量函数，`const` 全局变量，`inline` 函数，其他文件无法访问

Storage type:

- static data area: 全局变量，静态变量
- Stack: 局部变量
- Heap: 动态分配

storage class specifiers

- static
- extern
- auto: 告诉编译器这是一个局部变量
- register: 告诉编译器这个变量会频繁使用所以要保存到寄存器里面

Namespaces

Creating a namespace

只能出现在全局作用域，或嵌套在另一个命名空间

大括号后不需要 `;`

可以重命名 `namespace MyLib=Lib`

不能创建命名空间的实例

```

1 namespace mylib
2 {
3     //member
4     //friend
5 }
6 void main(){}
```

Using a namespace

Scope resolution

```

1 namespace X{
2     class Y
3     {
4         static int i;
5     };
6     class Z;
7 }
8 int X::Y::i=1;
9 class X::Z{
10     int u,v,w;
11     public:
12     Z(int i);
13 };
14 X::Z::Z(int i){u=v=w=i;}
```

Using directive

```

1 namespace cal{
2     double Add(double x,double y);
3     void Print(double x){cout<<x<<endl;}
4 }
5 using namespace cal;//直接引用
6 void main(){
7     double a=1,b=2;
8     Print(Add(a,b));
9 }
```

不要在头文件中直接用using directive

Using declaration

```

1 namespace cal{
2     double Add(double x,double y);
3     void Print(double x){cout<<x<<endl;}
4 }
5 void main(){
6     using cal::Add;
7     double a,b;
8     cal::Print(Add(a,b));//域运算符
9 }
```

Static members in C++

Static data members

```
1 class MyClass
2 {
3     static int obj;
4 };
5 int MyClass::obj=1; //静态成员必须在类的外面初始化，用类名限制，没有static关键词
```

在类中声明，所有对象共享，不属于对象，属于类

Static Member Functions

没有this指针，因此不能访问非静态成员，只能访问静态成员，可以做内联函数

Static initialization dependency

静态对象的初始化顺序保证与定义顺序一致，销毁顺序是逆序

Alternate linkage specifications--using C library

ch11 References & copy constructor

Pointers in C++

References in C++

创建引用时必须初始化，不能有NULL引用，初始化之后不能重新创建同名引用并初始化

```
1 int a=3,b=4;
2 int& m=a; //ok
3 int& m; //error
4 int& m=b; //error, m=b ok!m此时为b的值
5 int *p=&m;
6 int *&ref=p;
```

References in functions

```
1 int& g(int& x){ //int& x可以直接改变x的值，类似于指针操作
2     x++;
3     return x; //ok
4 } //返回int&, 代表返回的是一个变量，可以作为左值
5 int& h{
6     int q=0;
7     //return q; error: q作为局部变量无法返回
8     static int x=1;
9     return x; //ok
10 }
11 void main(){
12     int a=0;
13     g(a); //int*f(int*p); f(&a); 相似
14 }
```

Const references

让变量可以直接用常数赋值，而不一定用变量

```
1 void f(int& i){i++; }
2 void g(const int& j){cout<<j;}
3 void main(){
4     int a=1;
5     f(1); //error
6     f(a); //ok
7     g(1); //ok
8     g(a); //ok
9 }
```

Pointer references

修改指针的地址而不改变指针指向的值

```
1 increment(int* &i){i++;}
2 void main(){
3     int a=2;
4     int* p=&a;
5     increment(p);
6 }
```

The copy-constructor

```
1 Date today(2002,10,10);
2 Date tomorrow=today; //copy
```

除非用户自己定义，否则编译器使用默认复制

```
1 X::X(const X&)
```

```
1 class Date{
2     int y,m,d;
3     public:
4     Date(){} //constructor
5     Date(const Date& r)
6     {
7         y=r.y;m=r.m;d=r.d;
8     }
9     ~Date(){} //destructor
10 };
```

when copy:

- 用参数传递值
- 函数返回一个对象的值
- 引用传参不复制，因为没有创建新对象

Pointers to members

Pointers to data members

Pointers to member functions

ch12 Operator Overloading

语法

```
1 type operator @ (arg){  
2     //realization  
3 }
```

参数由一元/二元运算符和全局/局部变量同时决定

可重载的运算符

一元

- new, delete, new[], delete[]
- ++, --, (), [], +, -, *, &, !, ~

二元

- +, -, *, /, %, =, +=, -=, /=, %=
- &, |, ^, ^=, &=, |=, ==, !=, >, <, >=
- <=, ||, &&, <<, >>, >>=, <<=, ->, ->

Member functions

- prefix unary `a.operator @()`
- postfix unary `a.operator @(int)`
- binary `a.operator @(b)`

含有this指针

Friend functions

- prefix unary `operator @(a)`
- postfix unary `operator @(a, int)`
- binary `operator @(a, b)`

没有this指针

返回值

对于 `+/-/*` : `const T T::operator @(const T&c)`

对于 `+=/=/*=` 等赋值运算, 返回引用值: `T& T::operator @(const T&c)`

第一个运算对象如果是类的对象就用成员函数, 否则就用友元函数。

特殊运算符

下标[]

必须用**成员函数**重载。

ex: 对象可以直接通过下标访问数据

解引用->

必须用**成员函数**重载。

返回一个**指向类的指针**。

ex: 让类的对象能直接用->访问数据

->*

二元运算符

递增++递减--

可以前置也可后置，因此有四种函数

作为**成员函数**重载

以递增++为例：

prefix: `type operator++()`

postfix: `type operator++(int)`

注意返回值，前置直接返回递增后结果，后置需要返回递增前的结果。

不能重载的运算符

- `.`
- `.*`
- `::`
- `sizeof`
- `typeid`

成员和非成员之间的选择

Operator	Recommended use
All unary operators	member
<code>= () [] -> ->*</code>	must be member
<code>+= -= /= *= ^=</code> <code>&= = %= >>= <<=</code>	member
All other binary operators	non-member

赋值重载 Overloading assignment

```

1  x& x::operator=(const x& from)
2  {
3      //copy data from the argument
4  }
```

复制构造是对一个新对象的初始化，赋值重载是对两个已经存在的对象，把其中的一个赋值给另一个。在赋值重载或者赋值时注意如果类含指针成员时要主动定义，防止内存丢失。

```

1  class A{
2      int *p;
3  public:
4      A(int i){p=new int(i);}
5      ~A(){delete p;}
6      A(const A& r);//复制
7      A& operator =(const A& r);//赋值
8  };
9  A::A(const A& r){p=new int(*r.p);}
10 A&A::operator=(const A& r)
11 {
12     if(this==&r) return *this;
13     delete p;//删除原来的内存
14     p=new int(*r.p);//分配新内存
15     return *this;
16 }
```

类型转换

构造函数可以把一个double(1.5)临时转换成一个对象(1.5,0)

conversion operations:

```

1  #include<iostream>
2  using namespace std;
3  class complex{
4      double real,imag;
5  public:
6      complex(double r=0,double i=0){
7          real=r;
8          imag=i;
9      }
10     operator int(){return int(real);}//把double转换成int
11 };
12 void main(){
13     complex c(1.5,2.5);
14     cout<<int(c)<<endl;
15 }

```

<<和>>的重载

```

1  ostream& operator<<(ostream& ost,date& d)
2  {
3      ost<<d.year<<ends<<d.month<<d.day<<endl;
4      return ost;
5  }

```

ch13 Dynamic Object Creation

Object creation

当一个对象被创建时:

- 内存分配给对象
 - static storage
 - stack
 - heap(动态分配)
- constructor对内存初始化

Operator new/delete

```

1  Mytype *fp=new Mytype(1,2);
2  fp->print();
3  delete fp;

```

new/delete for arrays

```

1  Mytype *fp=new Mytype[100];//含一百个对象的数组,constructor全部初始为0
2  fp[10]=Mytype(10);//第十个重新初始为10
3  fp[10].print();//10
4  delete []fp;

```

Running out of storage

Overloading new/delete

ch14 Inheritance & composition

Composition syntax

```
1  class X{
2      int i;
3      public:
4      void set(int ii){i=ii;}
5  };
6  class Y{
7      int i;
8      public:
9      X x;
10     void f(int ii){i=ii;}
11 };
12 void main()
13 {
14     Y y;
15     y.f(10);
16     y.x.set(20);
17 }
```

Inheritance syntax

```
1  class X{
2      int i;
3      public:
4      X(){i=1;}
5      void set(int ii){i=ii;}
6      int permute(){
7          return i=i*10;
8      }
9  };
10 class Y:public X
11 {
12     int i;
13     public:
14     Y(){i=2;}
15     int change()
16     {
17         i=permute();//用X的函数并返回X中的i
18         return i;
19     }
20     void set(int ii)
21     {
22         i=ii;
23         X::set(ii);
24     }
25 }
```

```

25 }
26 int main(){
27     Y D;
28     D.change();
29     D.permute();
30     D.set(12);
31     return 0;
32 }

```

继承前需要先定义基类

Y继承自X,意味着Y会包含一个X的子对象

可以看出,当X的函数没有与Y中函数重名时可以直接引用(如permute),但是当派生类含同名函数时需要用 :: 标明是基类中的函数

基类: base class; 派生类: derived class

The constructor initializer list

新类的构造函数不能访问子对象的数据,构造函数和析构函数也不能被继承

在构造函数初始化列表中调用子对象的构造函数

```

1  class X{
2      int a;
3      public:
4      X(int i):a(i){}
5  };
6  class Y{
7      int b;
8      public:
9      Y(int i,int j):b(i),x(j){}
10     X x;//composition
11 };
12 class Y:public X
13 {
14     int b;
15     public:
16     Y(int i,int j):b(i),x(j){}//inheritance
17 };

```

Combining composition & inheritance

inheritance的子对象优先于composition,派生类的在最后初始化

声明顺序优先于初始化列表

Name hiding

在派生类只要定义一个与基类的同名函数,那么基类中所有的重载版本都会被隐藏

Functions not automatically inherit

构造函数, 析构函数, operator=

静态成员函数直接继承到派生类

```
1 class Base{
2     public:
3     static void print(){}
4 };
5 class Derived:public Base{};
6 int main(){
7     Derived::print();//可以直接调用
8     Derived d;
9     d.print();//也可以通过对象调用
10 }
```

静态函数只能访问静态数据

静态成员函数不能是虚函数

静态成员函数没有this指针

Choosing composition vs. inheritance

在只需要现有类的特点时选择composition,在需要相同接口时选择inheritance

Access control

public

如果类用public,那么派生类的成员函数可以访问基类的public和protected

派生类的对象可以访问public

private

派生类的成员函数可访问基类的public和protected

派生类的对象不能访问任意的基类成员

派生类的派生类不能访问

protected

派生类的成员函数可访问基类的public和protected

派生类的对象不能访问任意的基类成员

other rules

基类的private members只能被同类的成员或友元函数访问

只有基类的public members可以被基类的对象访问

派生类的成员只能访问基类的public和protected

派生类对象只有在继承public时才只能访问基类的public

Operator overloading & inheritance

除了赋值运算，其他运算符自动继承到派生类

Multiple inheritance

```
1  class A{
2      //...
3  };
4  class B{
5      //...
6  };
7  class C:access A,access B
8  {
9      //...
10 }
```

Disambiguity

问题：两个基类可能有同名的成员函数，并且如果这两个基类又都派生自另一个类可能会产生歧义

using ::

标明函数是来自哪个类

new function

在派生类中定义新函数，这个函数引用基类的函数

virtual base classes

B,C同时继承A,D同时继承B,C,解决当D用A函数时的歧义问题（D.B.A/D.C.A）,只需在B,C继承A时前加virtual就可以，这时A叫做虚基类

虚基类的构造函数只被调用一次，是被the most derived class 的构造函数调用的，虚基类的构造函数在所有非虚基类函数之前调用

虚基类构造函数必须在所有直接和间接派生类的构造函数初始化列表中列出，否则会调用默认构造函数

Upcasting

```
1  class Instrument{
2      public:
3      void play() const{}
4  };
5  class Wind:public Instrument{};
6  void tune(Instrument &i){i.play();}
7  void main(){
8      wind flute;
9      tune(flute);//upcasting
10     Instrument*p=&flute;//upcasting
11     Instrument &r=flute;//upcasting
12 }
```

ch15 Polymorphism & Virtual Functions

OOP特点: 封装, 继承, 多态

Evolution of C++ programmers

Upcasting

```
1  class Instrument{
2      public:
3      void play() const{
4          cout<<"Instrument::play"<<endl;
5      }
6  };
7  class Wind:public Instrument{
8      public:
9      void play() const{
10         cout<<"Wind::play"<<endl;
11     }
12 };
13 void tune(Instrument& i){
14     i.play();
15 }
16 void main(){
17     Wind flute;
18     tune(flute); //upcasting
19 }
```

the problem

我们想要引用Wind里面的函数, 但结果是引用了基类函数

virtual functions

```
1  class Instrument{
2      public:
3      virtual void play() const
4      {cout<<"instrument"<<endl;}
5  };
6  class Wind :public Instrument{
7      public:
8      void play() const //可以不写virtual
9      {cout<<"Wind"<<endl;}
10 };
11 void tune(Instrument&i){i.play();}
12 void main(){
13     Wind flute;
14     tune(flute);
15 }
```

虚函数是非静态成员函数

如果虚函数在类外定义, 那么 `virtual` 只需要在声明时使用

使用 `::` 时，虚拟的机制不会被使用

```
1 | void tune(Instrument&i){i.Instrument::play();} //这时用的是Instrument的函数
```

派生类的对应函数类型必须与基类的虚拟函数相同，否则只是一个新函数

实现多态行为必须满足：

- 派生类必须 `public` 继承基类
- 被调用的成员函数必须是虚函数
- 必须通过指针或引用操作对象

How C++ implements late binding

早绑定：多态失效

迟绑定：多态生效

Abstract base classes and pure virtual functions

有时你希望基类只为其派生类提供接口。你并不希望有人真正创建基类的对象。

一个纯抽象类只有纯虚函数

不能创建虚类的对象

当一个抽象类被继承时所有纯虚函数必须被实现，否则继承的类也成为抽象类

```
1 | class Point{//虚类
2 |     public:
3 |     Point(i=0,j=0){x=i,y=j;}
4 |     virtual void Set()=0;
5 |     virtual void Draw()=0;
6 |     protected:
7 |     int x,y;
8 | }
9 | class Line:public Point{
10 |     public:
11 |     Line()//...
12 |     virtual void Set(){
13 |         //具体实现
14 |     }
15 |     virtual void Draw(){
16 |         //具体实现
17 |     }
18 |     protected:
19 |     int x1,y1;
20 | }
21 | class Ellipse : public Point{
22 |     public:
23 |     Ellipse()//...
24 |     protected:
25 |     int x2,y2;
26 | } //仍然是抽象类（虚类），不能创建该类的对象
```

Inheritance and the VTABLE

每个包含虚函数的类都有一个虚函数表，派生类的表是基于基类的表改造的，重写后的虚函数换成自己函数的地址，没重写的沿用基类地址，新增的追加到表的后面

Object slicing

复制给新的对象时移除了现有对象的一部分

```
1 Dog d;  
2 Pet p=d; //导致对象切片，p无法使用只存在Dog中的函数  
3 Pet *p=&d; //可以  
4 Pet&p=d; //可以
```

Overloading & overriding

在派生类重写基类的一个重载函数，其他重载版本都会隐藏

已重写的虚函数的返回类型不能更改

Virtual functions & constructors

构造函数不能为虚函数

构造函数初始化虚指针

当一个包含虚函数的对象被创造时，虚指针必须初始化指向正确的虚表

Destructor and virtual Destructors

析构函数尽量声明为虚函数

当基类的析构函数为虚函数，派生类的自动成为虚析构函数

operator overloading

You can make operators virtual just like other member functions

Downcasting

unsafe

ch16 Template

Containers

Overview

Syntax

Function Templates

```

1  template <typename T1,typename T2>
2  T1 max(T1 x,T2 y)//比如T1是int, T2是double
3  {
4      //body
5  }

```

当实际应用时必须用具体类型填充，写的T1,T2必须全都用到

Class Templates

如果类的成员函数在类的外面定义，那么它必须定义为function template

```

1  template <typename T1,typename T2>
2  class Test{
3      public:
4          Test(T1 x,T2 y):a(x),b(y){};
5          void Print();
6      private:
7          T1 a;
8          T2 b;
9  };
10 template <typename T1, typename T2>
11 void Test<T1,T2>:: Print()
12 {
13     cout<<a<<b<<endl;
14 }
15 void main(){
16     Test<int,double> t1(10,1.2);
17     Test<char,int> t2('A',1);
18 }

```